

Tong Jia

510-676-7709 | San Diego, CA | jtpeter0829@gmail.com | github.com/peterjiaotong | linkedin.com/in/tong-jia/

EDUCATION

University of Southern California	Los Angeles, CA
Master of Science in Computer Engineering, GPA 3.7 / 4.0	September 2024 - May 2026
University of California, Santa Cruz	Santa Cruz, CA
Bachelor of Science in Computer Science, GPA 3.6 / 4.0	September 2020 - June 2024

TECHNICAL SKILLS

Languages:	Java, Python, JavaScript, TypeScript, C/C++, SQL
Backend & Data:	Spring Boot, Spring Security, Spring Data JPA / Hibernate, MyBatis, RESTful API Design, JWT, Node.js, PostgreSQL, MySQL, Redis, Elasticsearch, Flyway, Lombok
Frontend:	React, Vue 3, Ant Design, Vite, Zustand, Axios
AI & LLM:	LangChain, Spring AI, RAG, Tool Calling, MCP, ReAct
Cloud, DevOps & Tools:	Git, Docker, Kubernetes, Nginx, AWS, GCP, Tencent Cloud, Swagger/OpenAPI, Apache POI

WORK EXPERIENCE

Software Engineer Intern, LaunchPath LLC (Remote)	July 2026 - Present
- Building a native CRM system from the ground up for an early-stage AI-powered marketing platform, designing the core domain models for contacts, companies, and interaction records - Implementing a customer segmentation engine with composable filters over tags, behavioral sequences, and activity level, used directly to target and trigger email campaigns - Building a multi-agent marketing automation system with Spring AI: a Planner Agent breaks goals into subtasks, a Writer Agent retrieves the relevant data and generates the requested content, and an Optimizer Agent uses delivery data to adjust send strategy; with each agent running an internal ReAct loop and accessing the CRM through Tool Calling - Building a pgvector-based RAG knowledge base to improve agent output quality, where high-converting templates and industry best practices are embedded and retrieved as few-shot context at generation time; in A/B tests, AI-generated emails drew a 35% higher click-through rate than manual templates	
Software Engineer Intern, Centre Testing International Group Corporation (Remote)	May 2025 - June 2026
- Developed a production-grade bid-proposal generation platform using Apache POI and a custom template DSL for one-click .docx export with live editable previews, cutting document preparation time from ~2 hours to 3 minutes - Built a RAG assistant on LangChain4j and Milvus that lets environmental scientists query and interpret lab reports; combined sliding-window chunking, hybrid BM25 + vector retrieval, and re-ranking to lift retrieval accuracy ~25% over a pure semantic-search baseline - Designed metadata-driven RESTful APIs that let the frontend render each feature's UI dynamically, so onboarding a new business line requires only a JSON schema configuration with no additional frontend development - Implemented JWT authentication and RBAC authorization with Spring Security; deployed to Tencent Cloud (Nginx + systemd), achieving p95 latency < 110 ms at 50 concurrent sessions - Shipped a standards-compliance checker using regex and LLM adjudication to flag obsolete and superseded standards, adopted by 100+ colleagues in 3 months, cutting review time by 70% across 6,000+ reports and raising first-pass approval by 40%	
Research Assistant, AIEA Lab, UC Santa Cruz (Santa Cruz, CA)	April 2023 - May 2024
- Built an automated pipeline that scraped autonomous vehicle incident reports from the CA DMV and NHTSA and parsed 1,000+ unstructured PDFs into a unified, analysis-ready dataset - Quantified associations between incident variables and vehicle types using chi-square tests in SciPy, with Matplotlib and Seaborn visualizations incorporated into the AIEA Lab research paper	

PROJECTS

Mini-Spring: A Lightweight Spring Framework Implementation	Winter 2025
- Implemented a lightweight IoC container supporting bean registration, dependency injection, and full lifecycle management, including lazy loading, singleton and prototype scope patterns; resolved circular dependencies using three-level cache design - Built an AOP framework with before and after-returning advice, automatically selecting JDK dynamic proxy for interface-based targets and CGLIB for non-interface classes, executing interceptors via a chain-of-responsibility design pattern - Managed the full Bean lifecycle via Java reflection, parsing Bean definitions from XML configuration	